

DriftTrust Audit Code and Live Lab Guide

এই guide-এর লক্ষ্য: repository বুঝতে পারা, browser-এর ভিতরে model ও audit পর্যন্ত একটি operation অনুসরণ করা, এবং একজন visitor-কে সৎভাবে system দেখানো। The deployed product scores measured network flows and studies model adaptation with audit evidence. It does not automatically inspect the visitor's network or grant enterprise access.

GitHub Pages browser migration inspected 9 September 2026. The scientific engine and published initial weights remain from the measured experiment; the interface and persistence now run locally in the browser. Current primary site: GitHub Pages; model execution and session storage stay in the browser.

Read the visitor walkthrough first. Then follow the nine modules from architecture to evidence. The companion fieldbook contains interactive exercises; the published lab loads from GitHub Pages and runs its model in your browser.

Visitor walkthrough

1. Open the live lab. Wait for 'Real model ready · local browser storage connected'. Choose 'Audit-only · record failed checks' and click 'Start / reset model session'. A new isolated model session starts from the published checkpoint.
2. In 'Single flow', choose the actual MQTT_Publish example and click 'Run actual inference'. Look at the trust score, network-risk flag, model fingerprint and contributions. The example's known label is displayed for comparison but is not sent into the inference call.
3. Try DOS_SYN_Hping, then ARP_poisoning. The initial model correctly flags the supplied SYN example and misses the supplied ARP example. A reassuring score can be wrong. Each family has one supplied example, not a guarantee about all its flows.
4. Open 'Dataset replay' and click 'Process next 128 flows'. One browser request is handled as four internal batches of 32. Watch observed-flow and candidate counters. A button click supplies evidence; it does not command the model to adapt.
5. Click 'Run remaining replay'. 'Pause' stops after the current request finishes. There are 1,280 held-out measured flows in a constructed known / novel / known / novel order. The chart shows up to the latest 512 received flow scores.
6. Open the actual candidate records, inspect trigger, ECI, failed checks and model hashes, then click 'Export AJRs'. The prior hosted audit-only verification produced five accepted candidates. Counts describe that clean replay, not arbitrary uploaded inputs. The gated replay produces seven candidates: four activated and three discarded. Read the numeric check table before opening raw JSON. Verify record chain checks content and links; it does not establish independent truth.
7. To compare the gate, select 'Gated · veto failed model updates' and click 'Start / reset model session' again. Merely changing the dropdown does not change the active session policy. Replay from the initial checkpoint; compare decisions and flags. Export the old session first because Start/reset replaces this browser's active stored model session.
8. For your own data, use 'CSV stream' → 'Download CSV example'. Preserve all 12 column names and compatible measurement units. Replace rows with measured flows; optional label is 0 benign, 1 attack,

blank unknown. Then choose the file and 'Process CSV stream'. A nontechnical visitor can use supplied examples and replay without collecting traffic.

Live lab: https://the-sudipta.github.io/drifttrust_audit/lab.html

1 What is actually running

Three connected parts: evidence website, working browser lab, and reproducible experiment.

গল্প and motivation

গল্প / imagined scenario: একজন reviewer শুধু একটি সুন্দর dashboard দেখে জানতে চান—numbers কি button-এর scripted response, নাকি model-এর calculation? এই repository-তে সেই প্রশ্নের উত্তর code path অনুসরণ করে দেওয়া যায়।

সহজ Analogy

সহজ Analogy: index.html হলো museum-এর exhibition, lab.html হলো working laboratory, আর research scripts হলো experiment notebook. Exhibition results দেখায়; laboratory নতুন input হিসাব করে; notebook বলে results কীভাবে তৈরি হয়েছে।

Formal definition and code flow

The current public-data path is research/prepare.py → exported JSON model and streams → runtime/engine.mjs → research/run.mjs → research/report.py. GitHub Pages serves the model and code as static assets. runtime/browser-worker.mjs imports the identical engine and executes it in a browser background thread. The initial Worker API remains an optional separate server deployment in runtime/worker.mjs.

index.html + site.mjs + site.css render recorded results with dynamic metric and figure explanations. lab.html defines controls; lab.mjs reads user inputs and renders results. browser-client.mjs exchanges messages with browser-worker.mjs. That background worker performs real inference, detection, SGD, governance and IndexedDB persistence. No current lab operation needs an HTTP /api request or the former ChatGPT-hosted service.

The legacy Python path remains: main.py coordinates data/simulate.py and data/features.py, models/, drift/, governance/ and evaluation/. It is a synthetic telemetry prototype. Its TemporalMLPTrustScorer flattens a time window into an MLP; its name does not make it an LSTM. The live backend is a separate per-flow 12–24–1 MLP. Legacy label 1 means legitimate; current label 1 means attack. Do not mix their data or report their metrics as one experiment.

Math and worked application

1. Choose 'show paper results': index.html loads assets/research/results.json through site.mjs. No live model update is required.
2. Choose score this flow: lab.mjs calls `api("/api/infer", {x})`. The name is a message operation, not an HTTP fetch. browser-client.mjs posts it to a Web Worker; browser-worker.mjs reads IndexedDB state and calls `predict()/explain()` from engine.mjs.
3. Choose 'regenerate paper': prepare.py, run.mjs and report.py execute the offline protocol; changing the browser display does not regenerate experiments.

Interactive check

Before selecting an action, predict whether it needs only static files, a Worker request, or the offline experiment.

Select a task. The highlighted code path changes between static presentation, live Worker processing and offline experiment. Reset to paper results; no network calls are made.

Recall question: Which file supplies the same model equations to experiments and the working browser lab?

Answer: runtime/engine.mjs engine.mjs is imported by the offline run.mjs and browser-worker.mjs. worker.mjs remains a separate optional server API; main.py remains the legacy synthetic path.

Explain back: Explain the difference between the evidence page, live lab and old Python prototype.

Self-check: Mention static results versus newly computed predictions. Identify the shared engine, browser worker and local IndexedDB persistence. State that the legacy and current label conventions differ.

Source files: README.md, main.py, models/trust_scorer.py, research/run.mjs, runtime/worker.mjs, runtime/browser-worker.mjs, runtime/browser-client.mjs

2 A visitor's first ten minutes

Start with actual examples; use replay to see adaptation; upload measurements only when available.

গল্প and motivation

গল্প / imagined scenario: একজন visitor-এর কাছে CSV নেই এবং packet-rate মানে কী তাও জানেন না। তাঁকে শুরুতেই 12টি number বানাতে বলা উচিত নয়। Supplied held-out examples and replay give them a valid first route through the existing lab.

সহজ Analogy

সহজ Analogy: Single flow হলো একটি exam answer check করা; replay হলো একের পর এক answer দেখে শিক্ষক কোথায় ভুল করছেন তা বোঝা; labelled CSV হলো visitor-এর নিজের answer sheet, যেখানে correction থাকলে learning সম্ভব।

Formal definition and code flow

Start/reset creates an anonymous local model session after choosing a policy. No sign-up or server cookie is required. The active session is stored in this browser profile's IndexedDB and expires after 24 hours. Other tabs on the same site share it; other devices and browser profiles do not. Audit-only activates candidates while recording failed checks. Gated mode discards a candidate if any required check fails. Neither mode authorizes network access.

Single-flow inference reads the current model and returns a score and explanations. It does not increment observed-flow counters, change model weights, or produce an adaptation record. Editing a supplied example creates a hypothetical input; the displayed source label no longer validates the edited values.

Dataset replay supplies actual held-out records and reveals their labels after each 32-flow prediction batch. CSV stream uses visitor inputs. Unlabelled rows are scored and counted but do not provide training feedback. A batch without any labels cannot launch an update, even if its attributions change. Switching policy requires a new session, which also resets the replay cursor and model.

The stream chart now persists its latest 512 predictions and restores them on reload alongside counters. The point-inspection slider explains a selected flow's score, source and label. The detector explanation reports actual evidence and missing conditions. Raw JSON is secondary to the readable candidate decision table. Resetting clears this browser session's stored history; export first.

Math and worked application

1. Clean initial session: MQTT_Publish row 3502 gives attack score about 0.000038911; displayed trust rounds to 100.0/100. It is benign, but rounding does not imply zero risk.
2. Clean initial session: DOS_SYN_Hping row 21068 gives attack score about 0.999279; displayed trust is 0.1/100. The network-risk flag agrees with its attack label.
3. Clean initial session: ARP_poisoning row 13793 gives attack score about 0.195059 and trust 80.5/100. This is a false negative. These are locally recomputed checkpoint outputs, not invented teaching results.
4. After inference the stream counter is still zero. After a 128-flow replay request it becomes 128. Candidate count can remain zero because evidence and spacing conditions must pass.

Interactive check

Choose a visitor goal and whether trusted labels are available; predict which control to use and whether it can learn.

Choose example, replay or own CSV and toggle label availability. The recommended controls and learning eligibility update. Reset to first-time visitor; do not imply the offline guide contacts the live service.

Recall question: Why might 'Run actual inference' leave all adaptation counters at zero?

Answer: This endpoint scores one input without learning /api/infer is read-only with respect to model state. Replay or labelled stream processing supplies adaptation evidence.

Explain back: Guide a visitor from first opening the lab to exporting their first audit record.

Self-check: Start a session and use a real supplied example. Run replay and distinguish observed flows from candidates. Export records and reset before comparing policies.

Source files: lab.html, lab.mjs, runtime/worker.mjs, assets/research/examples.json

3 Data provenance and initial training

Real measurements support a bounded experiment, not every proposed enterprise claim.

গল্প and motivation

গল্প / actual project history: the public-data implementation adds UCI RT-IoT2022 to a project that previously used synthetic telemetry. It solves the need for measured flow inputs without pretending those flows contain identity, MFA or access-permission labels.

সহজ Analogy

সহজ Analogy: একটি বইয়ের identical photocopies train এবং test দুই জায়গায় থাকলে exam সহজ দেখাবে। Deduplication repeated input vectors সরায়; train / validation / test separation আলাদা কাজের জন্য আলাদা বই রাখে। Similar examples can still remain related.

Formal definition and code flow

prepare.py downloads the official archive, verifies its pinned SHA-256, assigns source-row IDs, selects 12 numeric features and maps benign family names to 0 and attacks to 1. It excludes invalid values, removes input-vector groups with conflicting binary labels, and deduplicates selected-feature vectors before splitting. Grouping uses pandas feature hashes; the study does not establish independence of related captures.

Observed counts: 123,117 raw rows, zero invalid selected numeric rows, 2,098 rows in nine conflicting-label groups removed, then 17,701 unique vectors retained. The 103,318 further removals are duplicates after conflict exclusion. Novel attack families ARP_poisoning, DDOS_Slowloris and NMAP_XMAS_TREE_SCAN are exclusively test data. Known-family partitions are 7,297 train, 2,432 validation and 2,433 test; 5,539 novel records join the shifted test, producing 7,972 records.

Only training data determines log1p means and standard deviations. Initial MLP fitting uses balanced training resampling and sklearn Adam; the best checkpoint is chosen by known-family validation log loss, up to 45 epochs with patience seven. Logistic regression and random forest are additional baselines. Validation supplies fixed canonical anchors; training supplies the initial replay buffer. Five fixed seeds are 11, 23, 37, 53 and 71; seed 11 is the predetermined published model.

No capture timestamps are released in this CSV. Known / novel replay phases are deliberately constructed. Novel means absent from initial train/validation, not an unknown zero-day designation. Labels are assumed available promptly during offline replay. RT-IoT2022 is a captured IoT testbed with generated attacks, not an enterprise access-control field trial.

Math and worked application

1. $123,117 - 2,098 = 121,019$ rows after conflict removal.
2. $121,019 - 103,318 = 17,701$ retained vectors.
3. $7,297 + 2,432 + 2,433 = 12,162$ known-family vectors.
4. $12,162 + 5,539 = 17,701$ total; $2,433 + 5,539 = 7,972$ shifted-test flows.

Interactive check

Predict which partition may supply scaler statistics, anchors and initial weights; assign each source before checking leakage.

Assign sources for scaler, anchors and initial weights. Report protocol leakage or valid selection. Reset to an intentionally incorrect assignment; use actual fixed partition counts.

Recall question: May the initial scaler use all test flows because their labels are hidden?

Answer: No, this protocol fits preprocessing on training only Using the held-out distribution would change the stated protocol. Here means, scales and initial fitting use training; validation selects checkpoints and anchors.

Explain back: Explain what makes this a real-data experiment and why it is not natural chronological drift evidence.

Self-check: Name the captured testbed and deduplication changes. Separate training, validation, known test and novel test. State that replay order and timely feedback are experimental assumptions.

Source files: research/prepare.py, assets/research/dataset.json, assets/research/RESEARCH_REPORT.md

4 How one flow becomes a score

Twelve numbers enter trained equations; the returned number is an uncalibrated attack score.

গল্প and motivation

গল্প / imagined scenario: একই packet count দুই device-তে একই risk নয়; duration, packet rate এবং payload pattern একসঙ্গে matter করে। A trained model combines these measurements rather than assigning a fixed number of points to each field.

সহজ Analogy

সহজ Analogy: 12টি observation-কে 24টি ছোট pattern detector দেখে, তারপর একটি final score তৈরি হয়। Analogy-র সীমা: hidden units have learned weights; they are not named rules like 'three SYN packets always mean an attack'.

Formal definition and code flow

transform() rejects vectors with wrong length or invalid numbers. For each feature it takes $\log(1 + \text{raw value})$, subtracts that feature's training mean, divides by its training standard deviation, then clips to -8 through 8 . The raw measurements retain their own units; transformed values are dimensionless.

forward() calculates 24 weighted sums plus biases, applies tanh, combines hidden outputs with another set of weights, and applies a numerically stable sigmoid. predict() returns the attack score p . The live display shows $100 \times (1 - p)$ as trust. $p \geq 0.5$ means FLAG_NETWORK_RISK, not deny access. These scores have not been calibrated to incident probabilities.

explain() sets one transformed feature at a time to zero, meaning the training mean in log space, and recomputes the score. Contribution = original attack score minus the altered score. Positive means the observed value raises the score relative to that background; negative means it lowers it. Contributions are perturbation comparisons, not SHAP, probabilities, causal effects or terms that must add to the prediction. The lab now displays all 12 contributions, with expandable explanations of each signed difference and the bar scaling.

The 12 fields are duration, forward/backward packet totals, packet rate, down/up ratio, SYN/RST/ACK counts, payload mean/std, and forward/backward initial window sizes. Field names and units must match the published collector semantics. A browser cannot obtain arbitrary network traffic merely by opening this page.

Math and worked application

1. Teaching arithmetic only: attack score $p = 0.80$. Trust fraction = $1 - 0.80 = 0.20$. Displayed trust = $0.20 \times 100 = 20/100$. The threshold flags network risk.
2. Teaching attribution only: original attack score 0.80 ; replacing one transformed feature yields 0.55 . Contribution = $0.80 - 0.55 = +0.25$. This compares two inputs under the same model, not an intervention on the network.

3. The interactive activity below embeds the actual published initial weights. Choose an actual held-out example, then edit a measurement. Edited combinations may be physically inconsistent or outside training support; they are sensitivity probes, not newly observed flows.

Interactive check

Before changing packet rate, predict whether this fixed trained model's score must rise. Then test it; nonlinear models need not be monotonic.

Choose a real example and change packet rate. Compute the actual frozen MLP locally, show score/flag and edited-input caveat. Reset restores the original row. Zero and large rates are boundary probes, not new measurements.

Recall question: Does a trust score of 80 mean the user has an 80% chance of being authorized?

Answer: No; it is 100 times one minus an uncalibrated flow attack score. The inputs describe traffic, not identities or permissions. No calibration or authorization policy supports that interpretation.

Explain back: Trace raw flow measurements through preprocessing, the MLP and the displayed trust score.

Self-check: Explain log transform and training-only normalization. Identify the 12–24–1 MLP and score threshold. Explain one signed contribution without claiming causality.

Source files: runtime/engine.mjs, runtime/worker.mjs, assets/research/model.json, lab.mjs

5 When the stream suggests drift

An attack flag concerns one flow; a drift signal concerns changing model behavior over evidence.

গল্প and motivation

গল্প / imagined scenario: একটি unusual flow হতে পারে attack, কিন্তু তা দেখেই classifier বদলানো ঝুঁকিপূর্ণ। A stream may instead contain a sustained unfamiliar pattern. The detector asks whether enough evidence suggests the current model is struggling or relying on features differently.

সহজ Analogy

সহজ Analogy: একজন শিক্ষক একটি answer ভুল check করলে training বদলান না। Recent answers-এ ভুল বেড়ে গেলে বা decision pattern বদলে গেলে review করেন। Detection asks for review; it does not prove why a change happened.

Formal definition and code flow

processBatch() first predicts every row using the same current model. It then incorporates supplied labels into an error history and a recent labelled buffer. The error detector needs 256 observed labels: compare the last 64 errors against the preceding 192. Trigger when the short error fraction is strictly greater than the reference plus 0.08. This is a fixed-window heuristic, not a full ADWIN implementation.

The second signal averages absolute perturbation contributions over the current batch, normalizes their total, and compares the largest component change to a stored reference. It triggers when that maximum difference is strictly above 0.15. The initial reference comes from validation anchors; after a candidate attempt it is refreshed using recent labelled records under the active model.

DriftTrust Audit Code and visitor guide

A candidate additionally needs at least 128 seen flows since the last attempt, at least 64 labelled recent records, and some label evidence in the current batch. A score-only stream cannot invent labels to meet those requirements. After an attempt, error history is cleared, spacing restarts, and the reference is refreshed even if the candidate was rejected. The counters and windows therefore do not simply accumulate forever.

No candidate does not mean no attacks. It can mean insufficient labels, no threshold crossing, insufficient spacing or use of single-flow inference only. The new lab shows actual per-batch detector evidence and explains why a candidate did or did not occur. Full error-history counts are shown before the post-attempt reset.

Math and worked application

1. Teaching counts: 12 errors in the recent 64 labels gives $12 / 64 = 0.1875$.
2. Teaching reference: 12 errors in the previous 192 gives $12 / 192 = 0.0625$.
3. $0.0625 + 0.08 = 0.1425$; $0.1875 > 0.1425$, so error drift is true once 256 labels exist.
4. If the gap is only 96 flows, the candidate is still blocked. Increase the gap to 128, retain 64 labelled recent records and current-batch feedback: an update attempt becomes eligible.

Interactive check

Predict whether the detector triggers and whether training is permitted; these are two different answers.

Change recent/reference error counts, attribution difference, available labels and spacing. Show signal and candidate eligibility separately using the actual thresholds. Reset to the worked case; equality does not pass strict drift thresholds.

Recall question: Attribution shift is 0.20 but the batch has no labels. Must the model adapt?

Answer: No, this implementation requires current label evidence Attribution can change on unlabelled inputs, but processBatch refuses a candidate without current label evidence and enough recent labels.

Explain back: Explain an observed zero-candidate count without assuming the service is broken.

Self-check: Distinguish flow classification from drift detection. Mention evidence, threshold and spacing requirements. Mention window resets after candidate attempts.

Source files: runtime/engine.mjs, assets/research/model.json, drift/detector.py

6 How a candidate learns

Training changes a copy first; the current batch is never rescored as if it had known its own labels.

গল্প and motivation

গল্প / imagined scenario: exam answers দেখে answer key মুখস্থ করার পর সেই একই attempt-কে 'আগেই জানতাম' বলা unfair। The evaluation avoids that ordering error by storing predictions before it exposes the batch's labels.

সহজ Analogy

সহজ Analogy: software patch staging—active program চলতে থাকে, copied version-এ change তৈরি হয়, তারপর checks run করে deploy decision নেওয়া হয়। Replay adds revision exercises so a candidate is not trained only on the latest examples.

Formal definition and code flow

The browser Web Worker divides a replay or CSV request into batches of at most 32 and calls `processBatch()` sequentially. Each batch's predictions are made first. Its labels then update evidence and can help train a candidate that affects subsequent batches. A 128-flow browser message contains four prediction-then-feedback cycles, not one model frozen across all 128 flows.

When eligible, the engine structured-clones the active model. `fit()` runs three SGD passes at learning rate 0.003 with a 0.0001 weight penalty. The ordinary adaptation modes use up to 128 recent labelled records plus up to 128 replay records. The `no_replay` ablation trains only on recent records. Initial sklearn Adam fitting and online JavaScript SGD are different stages.

For each row, forward propagation supplies hidden activations and p . The output error is $p - y$. Gradients are calculated, then output weights, hidden biases/input weights and output bias are adjusted. The resulting candidate is checked on identical validation anchors before any activation. This is real incremental weight training; no complete retraining is scheduled on every button click.

After an attempt the replay buffer appends recent records and keeps its last 128 entries. This is a rolling replay mechanism, not a guaranteed permanent memory of all old classes. Recent records can overlap with replay records on later attempts, so '256 training entries' does not necessarily mean 256 distinct flows.

Math and worked application

1. Teaching one-weight update, not the complete real network: $y = 1$, $p = 0.20$, hidden activation = 0.5, old output weight = 0.4, learning rate = 0.003.
2. Prediction error = $0.20 - 1 = -0.80$. Data-gradient contribution = $-0.80 \times 0.5 = -0.40$.
3. Weight penalty = $0.0001 \times 0.4 = 0.00004$. Total gradient = -0.39996 .
4. Change = $0.003 \times -0.39996 = -0.00119988$. New weight = $0.4 - (-0.00119988) = 0.40119988$. The other weights also change in the real engine.

Interactive check

Arrange prediction, feedback, candidate training, policy checks and activation in their valid order. Predict what is wrong with revealing labels first.

Use selects to place prediction, feedback, training, checking and activation in order. Show the first order violation and why it matters. Reset scrambles the steps; no retroactive prediction rewriting is allowed.

Recall question: Which predictions can a candidate trained after batch 1 affect?

Answer: Later batches after acceptance The batch's scores are saved before labels enter learning. Accepted weights are used for later calls; previous scores are not rewritten.

Explain back: Explain why offline and live results use a test-then-train sequence.

Self-check: Predict the whole 32-flow batch before feedback. Train a candidate copy using recent and replay entries. Apply accepted weights only to subsequent predictions.

Source files: runtime/engine.mjs, runtime/worker.mjs, research/run.mjs, tests/engine.test.mjs

7 Why a candidate is accepted or rejected

Audit-only documents checks; gated mode makes those same checks veto an update.

গল্প and motivation

গল্প / imagined scenario: নতুন attack চিনতে শেখা patch পুরোনো traffic-এ ভুল বাড়তে পারে। A fixed test set can detect that regression, but it can also prefer old behavior so strongly that useful new learning is blocked.

সহজ Analogy

সহজ Analogy: audit-only হলো inspection report সংরক্ষণ করে patch চালু করা; gated হলো mandatory inspection fail করলে patch আটকে রাখা। Passing a small checklist is not proof that every real-world requirement is satisfied.

Formal definition and code flow

The candidate and active model are evaluated on the identical fixed validation-anchor rows. Checks require ECI at least 0.5, anchor log-loss increase no more than 0.05, attack-anchor miss-rate increase no more than 0.05, and at least 128-flow spacing. The last condition is mostly enforced before training already. The policy mapping to ISO controls is a research proxy, not a certification assessment.

ECI compares anchor-based importance rankings before and after training. Top-5 Jaccard measures intersection divided by union; normalized Kendall tau-b measures ranking agreement with ties. ECI averages Jaccard and $(\text{tau} + 1) / 2$. A high ECI means explanations remained similar by these measures; it does not prove explanation correctness or fairness.

Audit-only (monitor_only) accepts candidates, retaining failed_checks and status ACCEPTED_WITH_FLAGS when needed. Gated (governed) activates a candidate only if all checks pass; otherwise status is REJECTED and the old model hash remains active. A rejected model candidate is not a blocked network request. Ungoverned, static, no_replay and no_attribution are offline comparison modes, not additional choices in the live dropdown.

Measured shifted F1 across five seeds: frozen 0.4060 ± 0.1239 ; audit-only and ungoverned 0.9064 ± 0.0191 ; gated 0.5872 ± 0.1600 . The adaptive pair has identical predictions by design; logging does not itself improve classification. Audit-only also has a higher benign false-alarm rate than the gate in this experiment. Gate sensitivity and anchor design remain open research work.

Math and worked application

1. Teaching ECI: two top-five sets share four features, so their union has six. $\text{Jaccard} = 4 / 6 = 0.6667$.
2. Teaching Kendall tau-b = 0.60. Normalize: $(0.60 + 1) / 2 = 0.80$. $\text{ECI} = (0.6667 + 0.80) / 2 = 0.7333$.
3. Teaching candidate: loss rises from 0.20 to 0.28, an increase of 0.08. ECI passes but the 0.05 loss limit fails.
4. Audit-only accepts this candidate with flags. Gated rejects it and keeps old weights. The calculation illustrates the policy; it is not an observed dataset event.

5. Actual screenshot reproduction: at flow 1,280, candidate 7 has ECI 0.94697 (passes). Anchor loss changes from 0.1132777 to 0.1922637: increase 0.078986, above the 0.05 limit. Attack-anchor misses improve from 8.333% to 6.25% (passes). Gated mode rejects because anchor loss alone fails. The entire gated compact replay has 7 attempts = 4 activated + 3 discarded.

Interactive check

Choose the mode, then vary ECI and anchor changes. Predict which single failure is enough to block a gated candidate.

Change ECI, loss/miss increases and policy mode. Show failed checks, activation/rejection and active model identity. Reset to the failed-loss example; equality passes gate limits.

Recall question: Does ACCEPTED_WITH_FLAGS mean every governance check passed?

Answer: No; audit-only accepted despite failed checks The status refers to activating a model candidate under audit-only policy. failed_checks records what did not pass.

Explain back: Explain the measured benefit and cost of gating without claiming it is always superior.

Self-check: Distinguish candidate acceptance from flow permission. Explain identical-anchor ECI and performance checks. Report lower shifted F1 for the current gate and its false-alarm tradeoff.

Source files: runtime/engine.mjs, assets/research/RESEARCH_REPORT.md, assets/research/results.json

8 Audit records and session persistence

The evidence describes actual candidate attempts and links each record to the previous one.

গল্প and motivation

গল্প / imagined scenario: model update-এর পরে reviewer জিজ্ঞেস করলেন 'কী শিখল, কোন records ব্যবহার করল, এবং কেন চালু হল?' শুধু final accuracy দিয়ে এই questions-এর উত্তর দেওয়া যায় না। AJR preserves the candidate decision context.

সহজ Analogy

সহজ Analogy: প্রতিটি page-এ আগের page-এর fingerprint রেখে notebook লিখলে মাঝের page বদলানো detectable হয়। কিন্তু কেউ পুরো notebook এবং শেষ fingerprint দুটোই নতুন করে লিখতে পারলে বাইরে রাখা trusted copy ছাড়া ধরা যায় না।

Formal definition and code flow

An AJR includes attempt and stream position, time, error/attribution trigger evidence, exact training and anchor IDs, model-before/candidate/active SHA-256 hashes, same-anchor losses and misses, ECI components, check booleans, failed checks, status, feedback source and incremental losses. It records each attempted adaptation, including rejection; it is not a complete log of all individual predictions.

canonical() sorts object keys for stable serialization; digest() hashes canonical JSON. Each record stores previous_record_sha256 and its own record_sha256; state stores chain_head. The verifier recomputes every hash and checks the links and head. Hashes identify content; they do not prove the labels were true, prevent all malicious activity, or let you reconstruct model weights from the hash alone.

DriftTrust Audit Code and visitor guide

browser-worker.mjs stores an active session ID, revision, expiry, model state, candidate AJRs and the latest 512 stream predictions in IndexedDB. Recent and replay buffers contain submitted feature values. No input is uploaded by the lab's model operations. The session is shared across tabs in the same browser profile; it is not a shared server database or centralized collection from all visitors.

The Web Worker serializes requests within its tab. Before committing a computed stream result, an IndexedDB read/write transaction rechecks the session ID and revision, preventing a stale tab from overwriting another tab's update or a reset. The latest request_id can return a saved reply; stale revisions return a conflict. Sessions expire after 24 hours and accept up to 20,000 flows. Unlike the old server API, the browser lab has no IP creation-rate limit. Local audit timestamps depend on the device clock; browser-controlled records are not independently trusted evidence.

Math and worked application

1. Record A has hash hA. Record B includes previous_record_sha256 = hA before its own hash hB is calculated.
2. Change B's status without changing its saved hash: recomputed digest no longer equals hB, so verification fails.
3. Rewrite both records and all their hashes: an exported head trusted elsewhere is needed to detect substitution of the whole chain.
4. Download session AJRs and run node scripts/verify-audit.mjs downloaded-session-audit.json. A successful check proves chain consistency relative to the provided head, not truth of every claim.

Interactive check

Verify a copied real experiment record, then change its status and predict whether the stored fingerprint still matches.

Verify a copied real primary-run AJR using canonical JSON SHA-256; edit status and reverify. Show mismatch and explain trusted-head limits. Reset restores the untouched copy; no original records change.

Recall question: Is an AJR created for every single-flow inference?

Answer: No; it records adaptation candidate attempts /api/infer does not train. AJRs are built inside processBatch when a candidate is trained and evaluated.

Explain back: Describe what an audit export proves and what it cannot prove.

Self-check: Mention exact evidence and model fingerprints. Separate hash-chain integrity from truth and immutability. Explain browser-profile isolation, shared tabs, 24-hour expiry and browser-controlled evidence.

Source files: runtime/engine.mjs, runtime/worker.mjs, db/schema.ts, scripts/verify-audit.mjs, assets/research/DEPLOYMENT.md

9 Reading evidence and defending the project

A working research system is valuable when its measured behavior and remaining scope are clear.

গল্প and motivation

গল্প / imagined reviewer conversation: 'Your demo runs, but does it establish general enterprise security?'
উত্তর দিতে implementation evidence, dataset findings এবং proposed integrations আলাদা করতে হবে। Strong explanations survive both a successful example and a known mistake.

সহজ Analogy

সহজ Analogy: test track-এ গাড়ি চালানো real engineering evidence; এটি সব রাস্তা ও আবহাওয়ায় accident-free driving-এর proof নয়। Likewise, measured IoT flows support a defined benchmark and live research experience.

Formal definition and code flow

research/report.py calculates attack-positive classification metrics, per-family behavior, rolling diagnostics and governance figures from stored predictions. Nine figures have PDF/SVG/PNG exports. Aggregate plots use five splits; detailed plots use seed 11. The online 1,280-flow compact replay is not the complete 7,972-flow shifted paper test.

Attack F1 combines attack precision and recall; it is not overall accuracy, calibrated probability or percent of all flows correct. The reported standard deviation describes variability over five overlapping dataset splits. The splits are not five independent real-world deployments. Stationary random forest F1 is strongest among these configurations; the adaptation advantage is conditional on the constructed shifted protocol.

tests/engine.test.mjs checks independent sklearn prediction parity, input rejection, ECI properties, prediction-before-feedback, no-label learning behavior, audit-only parity and rejected-update integrity. tests/api-check.mjs checks actual API behavior, isolation, retries, replay and chain consistency. The recorded hosted check processed 1,280 flows with five accepted audit-only candidates and maximum local/cloud replay-score difference about $8.33e-17$. These are behavioral checks, not penetration testing, enterprise certification or a complete browser-usability study. The GitHub Pages migration is checked separately for actual browser inference, replay parity, local persistence, CSV input and explained decisions; the earlier server verification file is historical evidence, not a test of this new hosting architecture.

For real deployment beyond this lab, an authorized flow collector must produce compatible measurements; trusted feedback must be connected; score calibration and delayed-label robustness need evaluation; identity and policy-enforcement integrations must be implemented if making access-control claims. The current UI exposes a real flow-risk model, not those unimplemented components. No tool can guarantee that reviewers accept the research paper.

Math and worked application

1. Teaching metric counts: true attacks correctly flagged = 80, benign flows wrongly flagged = 20, attacks missed = 40.
2. Precision = $80 / (80 + 20) = 0.80$. Recall = $80 / (80 + 40) = 0.6667$.
3. F1 = $2 \times 80 / (2 \times 80 + 20 + 40) = 160 / 220 = 0.7273$. This is not an assertion that 72.73% of every type of flow was classified correctly.

4. Suggested presentation: 'We trained on a public measured IoT dataset, replayed held-out flows with explicit feedback, and deployed the same inference/update engine. Visitors can inspect real predictions and model-change records. Gating reduced some errors but limited useful adaptation under this protocol.'

Interactive check

Change false positives and missed attacks while keeping correct detections fixed; predict which metric responds and what a visitor should infer.

Adjust TP/FP/FN counts and compute precision, recall and F1. Show each denominator and distinguish F1 from accuracy. Reset to 80/20/40; empty denominators return zero by a stated teaching convention.

Recall question: Which claim is supported by the present evidence?

Answer: Working flow-risk inference and auditable adaptation on a stated testbed protocol The source, measured runs and hosted checks support that bounded claim. Broader enforcement, certification and population-generalization claims require further evidence.

Explain back: Give a 60-second project explanation including one strength, one measured tradeoff and one unimplemented integration.

Self-check: Identify public measured data and the shared deployed engine. Explain inference → feedback → candidate → audit/governance. State the gate tradeoff and enterprise-enforcement limitation.

Source files: research/report.py, tests/engine.test.mjs, tests/api-check.mjs, assets/research/deployment_verification.json, assets/research/RESEARCH_REPORT.md

Repository map

File or directory	Responsibility
main.py; config.py	Legacy synthetic orchestration and settings; not the live API entry point.
data/simulate.py; data/features.py	Generate synthetic telemetry and window/normalize it for the legacy model.
models/trust_scorer.py; models/trainer.py	Legacy flattened-window sklearn MLP wrapper, initial fitting and persistence.
drift/detector.py; attribution.py; adapter.py	Legacy error/perturbation detection and continual replay adaptation.
governance/eci.py; policy.py; ajr.py	Legacy explanation consistency, policy proxies and JSON records.
evaluation/	Legacy baseline scores, metrics, governance summaries and charts.
research/prepare.py	Pinned dataset download, audit, split, train, export weights/streams/baselines.
research/run.mjs	Five seeds × two protocols × six engine modes, plus two static baselines.
research/report.py	Numerical summaries, nine scientific figures and result archives.

File or directory	Responsibility
runtime/engine.mjs	Preprocessing, MLP, attribution, error evidence, candidate SGD, ECI and chained AJRs.
runtime/worker.mjs	Optional earlier HTTP/D1 server API. The GitHub Pages lab does not call it.
db/schema.ts; drizzle/	Database schema/migrations for the optional server API; not required by browser IndexedDB.
index.html; site.mjs; site.css	Public research page, real results table, figure links and visual styles.
lab.html; lab.mjs	Live input forms, session controls, replay/upload actions and response rendering.
assets/research/	Published model, examples, replay, audit evidence, tables, figures and documentation.
scripts/build.mjs; scripts/dev.mjs	Bundle Worker/static site; local server with SQLite D1 adapter.
scripts/package-site.ps1; .openai/hosting.json	Windows packaging and Sites project/database binding metadata.
scripts/verify-audit.mjs; tests/	Hash-chain verification, scientific engine checks and API integration checks.
package.json; pnpm-lock.yaml; requirements-research.lock.txt	Commands and locked Node/Python dependencies for reproducibility.
.github/workflows/; informations/	Automated checks and historical methodology/status documents. Historical claims are not substitutes for the current report.
runtime/browser-worker.mjs	Actual browser-side model operations, evidence explanations and IndexedDB transactions.
runtime/browser-client.mjs	Message bridge from lab UI to the Web Worker; operation names resemble API paths but do not send HTTP requests.
explanations.mjs	Feature meanings, metric definitions, precise audit-decision arithmetic and field dictionary.
scripts/build-pages.mjs; scripts/serve-pages.mjs; .github/workflows/pages.yml	Build static GitHub Pages assets, preview the subpath locally, and deploy through GitHub Actions.

Browser operation map

Route	Purpose
/api/health	Model and database availability

Route	Purpose
/api/model	Feature schema, examples, published configuration
/api/session	New audit-only or gated session
/api/session	Current counters, mode, revision, cursor
/api/infer	One x vector; read-only prediction and explanation
/api/replay	revision, request_id, count; published held-out records loaded as static JSON
/api/stream	revision, request_id, rows with x and optional y
/api/audit	Session candidate records and chain head

Every page control and display

Control or text	Meaning and consequence
Navigation and DT mark	Research results returns to evidence; Full interactive guide opens the offline-capable study tool; Code flow jumps to the operation trace; Source code opens GitHub. DT is branding, not a metric.
Runtime status and dot	Loading means assets/storage are not ready; ready means trained weights and local IndexedDB are accessible. The green dot is availability, not a safety rating.
Policy dropdown	Chooses the policy for the next reset session. It does not silently change the current session. The adjacent text warns when the selected policy differs from the active one.
Start / reset model session	Loads initial weights and replaces the browser's current model state, replay position and records. Export first.
Session revision and labels	Revision counts completed stream operations; it is not the candidate count. Labels received counts rows with known/supplied 0/1 feedback.
Single flow tab	Shows the one-flow prediction form. Switching tabs changes the visible panel, not model weights.
Held-out example dropdown	Loads an exact measured source row and shows its original label/family. It does not run inference until requested. Edited values invalidate that reference label for the modified input.
Feature fields and meaning panels	All twelve fields show their unit, semantic definition, current value and training min/max. These bounds describe observed training data, not universal physical limits or a correctness guarantee.

Control or text	Meaning and consequence
Run actual inference	Scores one input with current weights and explains all features. No training, stream increment or AJR.
Dataset replay tab	Shows the compact 1,280-flow constructed sequence. The supplied dataset labels are revealed after prediction, enabling supervised updates.
Regime boxes and progress bar	Four 320-flow known/novel/known-return/novel-return phases. Highlighting is data mix; it is not detector ground truth. Progress is processed replay rows divided by 1,280.
Process next 128 flows	Runs four consecutive internal batches of 32. An accepted update can affect later batches within this request.
Run remaining replay	Repeats 128-flow requests until the replay ends or Pause is requested. It does not guarantee an update after each request.
Pause after this batch	Stops before the next request after the current one completes. It does not roll back results or kill a half-committed update.
CSV stream tab and file chooser	Selecting a file shows name and size. It does not calculate or upload measurements. Exact header names map the twelve features, independently of CSV order.
Process CSV stream	Validates all rows, then computes locally. Maximum 256 rows and 2 MB. Optional labels use 0/1/blank. Unlabelled uploads cannot initiate training.
Download CSV example	Saves a header and one measured example as a starting format. Replace it with your measurements, not invented credentials or identity fields.
Trust score and risk badge	Trust is 100 times one minus the raw attack score. The badge flags attack score at least 0.5. The dynamic paragraph shows arithmetic and correctness against a valid supplied label. Neither is an authorization decision.
Model fingerprint panel	SHA-256 identifies current weight content. A changed hash means changed content, not improved performance.
Input Predict Detect Candidate Audit buttons	Selecting any stage explains its code function. Green indicates stages involved in the last operation; an amber audit stage indicates flagged checks. These colors are not user access permissions.
Four counters	Observed stream flows; candidate models trained; candidates activated; candidates discarded. Attempts equal activated plus discarded. Single inference does not increment them.
Stream chart and point slider	Y is trust 0–100; X is flow position. Dashed line is 50. A point is one prediction. The slider explains exact source, label and score. Lines connect record order, not elapsed time. Latest 512 values persist and restore.

Control or text	Meaning and consequence
Why an update happened panel	Shows actual last-batch error-history size, recent/reference error, attribution shift, label count and spacing. It distinguishes a detector signal from eligibility for candidate training.
Feature contribution rows	All twelve signed perturbation comparisons are ranked by absolute size. Expand to read each meaning. Bar width is relative to the largest contribution for this input, not a probability.
Refresh saved records	Reads current IndexedDB audit records without training or changing decisions.
Verify record chain	Recomputes record hashes, links and chain head. It checks internal consistency, not ground-truth labels or independent immutability.
Export AJRs	Downloads the actual local session audit as JSON. No new model update occurs and the download is not a server submission.
Audit filter	Shows all, activated or discarded candidates. Filtering never modifies the stored audit or model.
Candidate row header	Candidate number, processed-flow position, activated/discarded state and ECI. Expand to see the exact reason and old/candidate comparison.
Candidate check table	Each check shows old value, candidate value, threshold, pass/fail and an explanation of the arithmetic. One failing check rejects a gated candidate, even with a high ECI.
Raw audit field dictionary and JSON	Every top-level field has a definition. Training IDs can repeat through replay; top-feature IDs are zero-based indices. Raw JSON is for independent verification, not the first explanatory view.
Chain head	The newest record hash links the end of the audit. Keep a trusted copy elsewhere if detecting substitution matters.
Code-flow operation selector	Changes the displayed function/file path and explanation for inference, replay, audit or initial research fitting. It is an explanation selector, not an execution button.
Error messages	Describe missing sessions, invalid values/files, stale revisions or load/storage failures. A technical error is not the same as a risky flow or rejected candidate.
Research protocol selector	Chooses recorded stationary versus shifted results. It does not run new experiments.
Research table configuration buttons	Select a model to explain below the table. They do not activate that configuration in the live session.
Configuration and metric explanation selectors	Dynamically translate the selected measured mean/SD, model method, metric formula and scope. F1 is not accuracy; false-alarm rate uses benign flows as its

Control or text	Meaning and consequence
	denominator.
Research figures and legends	Each figure's expansion explains axes, colors, aggregation, baseline/reference lines, primary-seed versus five-run scope and interpretation limits. Workflow arrows indicate operations, not causal proof.
PDF SVG PNG and archive links	Download static paper artifacts. PDF/SVG are vectors; PNG is raster. The figure archive contains 27 files; the numerical archive contains predictions and event results. Downloading does not train.
Model manifest dataset and source links	Open reproducibility files: fitted weights/scaler, actual data counts/checksums, execution provenance and source code. They are evidence locators, not endorsements or certifications.
Fieldbook module navigation and three stages	Select a topic, then Understand, Work it out or Explain it. Activities compute locally and never mutate a live model session.
Fieldbook Reset and quizzes	Reset restores illustrative exercise inputs. Quizzes give explanatory feedback and allow retry; they do not evaluate research validity.
Fieldbook explain-back and completion	Write your own explanation, reveal a checklist, and explicitly self-mark completion. No AI grades it. Drafts remain in memory until reload.
Review deck and note export	27 cards can be revealed, self-graded, shuffled, filtered to আবার and restored. Export notes downloads drafts/progress; reload resets study memory.
Glossary search and Close	Filters local definitions. Close or Escape returns to the lesson. No search query leaves the device.
Footer attribution and CC BY 4.0	Names the dataset creators/license. It is reuse attribution, not evidence of security certification.

All twelve input attributes

flow_duration “Flow duration

seconds. Time from the beginning to the end of this measured bidirectional flow. Longer duration can mean a long connection, but does not by itself mean an attack.

fwd_pkts_tot “Forward packets

packets. Packets in the flow's forward direction, from its originator toward its responder.

bwd_pkts_tot “Backward packets

packets. Packets travelling back from the responder toward the originator. Zero means none were counted in this direction.

flow_pkts_per_sec “Flow packet rate

packets / second. How quickly packets occurred over the measured flow. Bursts and short durations can produce large rates. High rate alone is not a universal attack rule.

down_up_ratio “Down/up ratio

ratio. Backward-to-forward packet ratio in the dataset’s flow representation. A value of 1 indicates equal directional packet counts. Preserve the collector’s zero-denominator convention.

flow_SYN_flag_count “SYN flags

count. Packets carrying the TCP SYN flag, used in connection synchronization. It is a count, not a yes/no attack indicator.

flow_RST_flag_count “RST flags

count. Packets carrying the TCP RST flag, which resets a TCP connection. Resets can occur in both ordinary traffic and attacks.

flow_ACK_flag_count “ACK flags

count. Packets carrying the TCP ACK flag, acknowledging received TCP data or connection state.

flow_pkts_payload.avg “Payload mean

bytes. Average packet payload size: the data portion measured by the flow collector, rather than a count of packets.

flow_pkts_payload.std “Payload standard deviation

bytes. How much packet payload sizes vary around their mean. Zero indicates no variation among the measured payload sizes.

fwd_init_window_size “Forward initial window

bytes. Initial advertised TCP receive-window size in the forward direction. Zero can occur in records without a measured TCP window; it does not prove an attack.

bwd_init_window_size “Backward initial window

bytes. Initial advertised TCP receive-window size in the backward direction. Keep the measurement semantics consistent with the training data.

All configured model settings

batch_size = 32

Maximum flows in one prediction-before-feedback cycle. A 128-flow click contains four 32-flow cycles.

error_short = 64

Latest labelled prediction errors used for the recent error fraction.

error_long = 256

Total labelled errors required for an error trigger. The older reference uses $256 \hat{=} 64 = 192$ labels.

error_margin = 0.08

Required absolute rise in recent error fraction: 0.08 means 8 percentage points. The drift comparison is strictly greater than this margin.

attribution_threshold = 0.15

Maximum absolute change in normalized mean absolute feature contributions must exceed 0.15. This is a feature-importance shift, not a probability of drift.

min_update_gap = 128

Minimum processed flows between candidate training attempts, including rejected attempts.

adaptation_window = 128

Maximum recent labelled entries retained for candidate training. At least 64 recent labels and some current-batch labels are also required.

replay_size = 128

Maximum older labelled entries mixed into candidate training to retain previous behavior. This training buffer differs from the 1,280-flow Dataset replay tab.

update_epochs = 3

Number of sequential SGD passes over the recent-plus-replay candidate training set.

update_lr = 0.003

Learning rate: scale applied to each SGD gradient step. Weight regularization uses a fixed coefficient 0.0001.

eci_threshold = 0.5

Minimum Explanation Consistency Index for a passing check. $ECI = (Jaccard + (Kendall\ tau + 1) / 2) / 2$. Jaccard compares the top five features; tau compares all feature rankings.

max_anchor_loss_increase = 0.05

Largest permitted candidate-minus-old binary log-loss increase on identical fixed validation anchors. Lower loss is better; equality passes.

max_attack_miss_increase = 0.05

Largest permitted rise in missed-attack fraction on attack-labelled anchors. 0.05 means 5 percentage points; equality passes.

Every audit-record field

schema_version

Format of this record, not model quality.

engine_version

Version name of the shared inference/update implementation.

attempt

Number of the candidate training attempt in this session.

position

Number of stream flows processed when this candidate was checked.

timestamp_utc

Time recorded for the attempt; in the browser lab it uses this device's clock.

trigger

Which signal initiated the attempt; error shift takes precedence when both signals trigger.

signals

Measured short/reference errors and maximum normalized attribution change.

training_record_ids

Exact entries used for candidate learning; replay overlap can repeat IDs.

anchor_record_ids

Fixed validation-record IDs used to compare both models.

model_before_sha256

Fingerprint of the active weights before training the candidate.

candidate_sha256

Fingerprint of the newly trained candidate weights.

active_model_sha256

Fingerprint of the weights retained after the decision.

same_anchor_before

Old model's loss, attack misses and mean trust on fixed anchors.

same_anchor_candidate

Candidate's values on the identical anchors.

explanation_consistency

ECI, top-five overlap, Kendall ranking agreement and ranked feature indices.

checks

Pass/fail booleans for the configured research governance rules.

policy_mapping

Research associations with ISO controls; not a certification.

mapping_scope

Explicit limit on the policy mapping claim.

accepted

Whether candidate weights became active, not whether a flow was permitted.

status

ACCEPTED, ACCEPTED_WITH_FLAGS or REJECTED model update.

failed_checks

Checks that failed, even if audit-only still accepted the update.

incremental_losses

Training log loss reported for each of the three SGD passes; not held-out performance.

previous_record_sha256

Link to the preceding record's content fingerprint; null for the first.

record_sha256

This record's canonical-JSON SHA-256 fingerprint, excluding this field itself.

feedback_source

Where labels came from; visitor assertions are not independently verified.

elapsed_ms

Measured candidate-processing time on the executing device; not a controlled latency benchmark.

How to read all nine research figures

01_dataset_audit " Dataset composition

Raw versus unique input vectors by family. The family names identify dataset traffic categories. The horizontal count axis is logarithmic: equal spacing represents multiplication rather than equal additions. Paired bars compare raw rows with unique retained feature vectors after conflict exclusion. Deduplication drastically changes prevalence; these are record counts, not classifier performance.

02_baselines_and_ablations " Baselines and ablations

Eight configurations, two protocols, five seeded splits. The vertical axis is attack F1 from 0 to 1. Bars group model configurations under stationary and shifted replay. Each height is the mean of five runs; error bars are one sample standard deviation. Compare bars within the same protocol. Audit-only and ungoverned predictions are identical by design; logging alone does not improve classification.

03_stream_performance " Performance through the stream

Primary-seed rolling balanced accuracy and error fraction. The horizontal axis follows flow position in the constructed primary-seed stream. Values summarize nonoverlapping 256-flow reporting windows. Balanced accuracy treats both classes equally; error fraction counts all wrong classifications in that window. Dotted vertical boundaries mark deliberately constructed data regimes, not detected events or timestamps. The plot is not five-seed aggregate performance.

04_roc_pr ROC and precision-recall

Primary-seed comparisons across score thresholds. ROC uses false-positive rate horizontally and attack true-positive rate vertically. Precision-recall uses attack recall horizontally and precision vertically. Curves vary the decision threshold; the normal deployed threshold is 0.5. The PR baseline is the observed attack prevalence. These diagnostics use primary-seed shifted predictions; high curve area does not prove probability calibration or every-family detection.

05_confusion_matrices Class-specific errors

Counts and within-true-class percentages. Axes distinguish the true label from the predicted label: 0 benign, 1 attack. Diagonal cells are correct. A true attack predicted benign is a false negative; a true benign flow predicted attack is a false positive. Each percentage is normalized within its true class, not over the entire dataset. Colors visualize magnitudes and do not signify governance acceptance.

06_governance_decisions Accepted and rejected model candidates

ECI and anchor loss for actual primary-seed gated attempts. Each point is one candidate model attempt, not a network user. ECI compares explanation rankings on fixed anchors. Anchor log-loss change compares candidate and old predictions on the same anchors. Dashed lines are the preselected policy limits. Green circles activate model versions; red crosses discard them. Passing ECI alone is insufficient when another check fails.

07_per_family Performance by traffic family

Correct classifications and actual held-out support. Each category is a dataset family; support tells how many held-out examples contribute. The rate is correct classification within that family, not a separate calibrated probability. Tiny support can make rates unstable. Novel families were absent from initial fitting, but their labels may influence later prequential updates after their predictions.

08_research_workflow Experimental workflow

Preparation, disjoint split, model selection and evaluation. Boxes name computational stages and arrows indicate data/control flow. Download and conflict/deduplication audit precede splitting. Training determines preprocessing and initial weights; validation selects checkpoints and anchors. Held-out streams are predicted before feedback-driven updates. Five seeds and two protocols lead to metrics and figures. Diagram arrows are workflow dependencies, not empirical causal evidence.

09_live_architecture GitHub Pages and browser execution architecture

Actual inference, candidate checks and local evidence storage. Boxes identify the browser inputs, Web Worker running the shared model engine, fixed-anchor checks and local IndexedDB storage. Arrows trace predictions and candidate model updates. PASS / AUDIT-ONLY means a passing candidate is activated, or audit-only policy activates it despite flags. FAIL + GATED means the candidate weights are discarded and the previous active model remains. These branches govern model versions, not user access. GitHub Pages serves the files; your browser performs inference and learning. Local audit chains require a separately trusted saved head to establish evidence beyond browser-controlled consistency.

Developer commands and safe interpretation

`pnpm install --frozen-lockfile` → install the Node dependency lock. `pnpm build:pages` then `pnpm dev:pages` → local static lab at http://localhost:8788/drifttrust_audit/lab.html with browser IndexedDB state. `pnpm dev` retains the optional server API for separate API tests.

`python research/prepare.py` → retrieve/check data and fit initial models; `node research/run.mjs` → execute streaming comparisons; `python research/report.py` → regenerate metrics and figures. Install `requirements-research.lock.txt` in Python 3.12 first.

`pnpm test` → scientific engine tests. With the local server running, `node tests/api-check.mjs` <http://localhost:8787> → API behavior checks. `pnpm build` → static assets and Worker bundle. Build does not itself publish.

`node scripts/verify-audit.mjs downloaded-session-audit.json` → recompute canonical hashes and verify chain links. A trusted saved head strengthens later substitution detection.

The original `main.py` is a separate synthetic workflow and clears files in its configured `audit/results` output directories at startup. Do not confuse running it with starting the live service.

Troubleshooting the visitor experience

Service unavailable: open GitHub Pages or run `pnpm build:pages` and `pnpm dev:pages`; double-clicking `lab.html` alone cannot load its browser module worker and JSON assets. The offline fieldbook is intentionally different and works without a server.

No candidates: ensure you used replay or a labelled stream, supplied enough labels and met spacing/trigger rules. One suspicious flow does not force retraining.

Policy appears unchanged: select a policy and then start a new session. A selection alone does not mutate the existing session.

Unexpected score: compare known truth separately, inspect training-range warnings and remember scores are uncalibrated. Edited example values no longer have a validated dataset label.

CSV rejected: check exact header names, 12 numeric features, non-negative finite values, 0/1/blank labels, maximum 256 rows and 2 MB browser file size. The optional older server API has a separate 400 KB JSON body limit; the browser lab sends no model inputs to it.

Session expired or conflict: create a new session after expiry; a 409 requires refreshing current revision. Export useful evidence before replacing the session. The last-request retry protection is not an unlimited request-history cache.

Glossary

Flow — একই bidirectional network conversation-এর measurements; not a login identity.

Inference — বর্তমান weights দিয়ে নতুন input-এর score হিসাব করা.

Label — Feedback: current backend uses 0 benign, 1 attack, null unknown.

Drift — Changing error or attribution behavior across evidence; not synonymous with one attack.

DriftTrust Audit Code and visitor guide

Candidate — A trained model copy awaiting activation policy.

Anchor — Fixed validation record used to compare old and candidate models fairly.

Replay buffer — Older labelled entries mixed into an incremental update; bounded rolling memory.

ECI — Explanation Consistency Index, based on ranking similarity across identical anchors.

AJR — Adaptation Justification Record, evidence about one attempted model update.

Revision — Session version number used to detect conflicting writes.

F1 — A score combining positive-class precision and recall; not overall accuracy.

False negative — An attack that receives the lower-risk classification.

False positive — A benign flow that receives the network-risk flag.

Calibration — Whether score values correspond reliably to observed event frequencies; not established here.

Hash chain — Content fingerprints linked across records; useful with a separately trusted head.

IndexedDB — This browser profile's persistent local database; clearing site data deletes local sessions.

Web Worker — Background JavaScript execution keeping real model computation off the page's main UI thread.

D1 — Database used by the optional earlier server API, not the primary GitHub Pages lab.

Sources and claim boundaries

Current repository and browser migration: https://github.com/the-sudipta/drifttrust_audit. The scientific engine remains the same as the original measured snapshot. Module source-file locators identify the implementation behind each explanation.

Dataset: B. S. Sharmila and Rohini Nagapadma. RT-IoT2022. UCI Machine Learning Repository. DOI 10.24432/C5P338. CC BY 4.0. <https://archive.ics.uci.edu/dataset/942/rt-iot2022>

Measured findings: `assets/research/RESEARCH_REPORT.md`, `dataset.json`, `results.json` and `deployment_verification.json`. This dossier adds explanation, not a new benchmark run. Actual initial-model example outputs were recomputed directly from the published checkpoint for this guide.

Teaching scenarios and policy/metric arithmetic are explicitly illustrative. The embedded score activity uses real frozen weights; editing a feature creates a sensitivity probe. The offline guide never modifies a lab model session or submits measurements. Its audit activity edits only a copied record.